

IsingMachine

September 19, 2024

0.1 Checking GPU on Mac

0.2 Using Tensorflow and Diagnosis of the Performance and Optimization of the model

```
[2]: import tensorflow as tf
from tensorflow.python.profiler import profiler_v2 as profiler
import time
import numpy as np
import matplotlib.pyplot as plt
from IPython.display import display, Markdown
```

```
[3]: # List physical devices available
gpus = tf.config.list_physical_devices('GPU')

# Print the number of GPUs detected
print(f"Number of GPUs available: {len(gpus)}")

# Optional: print detailed info about each GPU
for gpu in gpus:
    print(gpu)
```

Number of GPUs available: 1

PhysicalDevice(name='/physical_device:GPU:0', device_type='GPU')

```
[4]: # import tensorflow as tf

# # Specify the directory to save the profiling data
# logdir = 'profiler_logs'

# # Start profiling
# tf.profiler.experimental.start(logdir)

# # Run your TensorFlow code
# with tf.device('/GPU:0'):
#     a = tf.constant([[1.0, 2.0, 3.0]])
#     b = tf.constant([[1.0], [2.0], [3.0]])
#     result = tf.matmul(a, b)
#     print(result)
```

```
# # Stop profiling
# tf.profiler.experimental.stop()
```

```
[5]: # Load TensorBoard to visualize the profiling results
%load_ext tensorboard
```

```
[6]: # %tensorboard --logdir=profiler_logs
```

0.3 Using GPU T4 from Kaggle

The T4 GPU provided by Kaggle is well-suited for most deep learning tasks, offering a balance between performance and energy efficiency. It is particularly good for inference and small-to-moderate training jobs.

The T4 GPU in Kaggle has 16 GB of GPU memory, which is enough for training many types of models (e.g., CNNs, transformers). However, for very large datasets or complex models, memory might become a limiting factor.

1 Spatial Photonic Ising Machine By Spatial Light Modulation

A spatial light modulator (SLM) is a device that can manipulate the phase, amplitude, or polarization of a light beam spatially. In other words, it can control the properties of light pixel by pixel across its surface. The SLM then applies the corresponding phase shift to each pixel based on the desired spin configuration.

SLMs can modulate the phase of light by controlling the thickness of a liquid crystal layer pixel by pixel. By applying different voltages to each pixel, the refractive index of the liquid crystal can be changed, altering the phase of the light passing through.

1.1 Procedure

1.2 Randomly Generate a Phase Matrix

```
[7]: def generate_random_phase_matrix(n):
    """
    Generates a random phase matrix of size nxn.
    """
    # Generate random phases (0 or )
    phase_matrix = np.random.choice([0, np.pi], size=(n, n))
    return phase_matrix
```

1.2.1 1. Encoding spin variables:

Convert binary spin values (+1, -1) into phase modulations of a light field using a spatial light modulator.

$$\begin{bmatrix} 0 & 0 & \pi & 0 \\ \pi & 0 & 0 & \pi \\ 0 & \pi & \pi & 0 \\ \pi & 0 & 0 & \pi \end{bmatrix} \rightarrow \begin{bmatrix} 1 & 1 & -1 & 1 \\ -1 & 1 & 1 & -1 \\ 1 & -1 & -1 & 1 \\ -1 & 1 & 1 & -1 \end{bmatrix}$$

```
[8]: def binary_phase_to_spin(phase_matrix):
      """Converts a binary phase matrix to a spin variable matrix.
      """
      spin_matrix = np.where(phase_matrix == 0, 1, -1)
      return spin_matrix
```

```
[9]: def print_phase_matrix_in_pi(phase_matrix):
      """Prints the phase matrix with values displayed as '0' or ' '."""
      phase_symbols = np.where(phase_matrix == 0, '0', ' ')

      # Convert the matrix to a readable string format
      for row in phase_symbols:
          print(' '.join(row))
```

```
[10]: # Example usage
phase_matrix = np.array([
    [0, 0, np.pi, 0],
    [np.pi, 0, 0, np.pi],
    [0, np.pi, np.pi, 0],
    [np.pi, 0, 0, np.pi]
])
```

```
[11]: spin_matrix = binary_phase_to_spin(phase_matrix)
```

```
[12]: print("Phase Matrix:")
print_phase_matrix_in_pi(phase_matrix)
print("Spin Matrix:")
print(spin_matrix)
print(f"The shape of the phase matrix is {phase_matrix.shape}.")
print(f"The size of the phase matrix is {phase_matrix.size}.")
```

Phase Matrix:

```
0 0  0
 0 0
0  0
 0 0
```

Spin Matrix:

```
[[ 1  1 -1  1]
 [-1  1  1 -1]
 [ 1 -1 -1  1]
 [-1  1  1 -1]]
```

The shape of the phase matrix is (4, 4).

The size of the phase matrix is 16.

1.3 Now construct the Target spin matrix and the random spin matrix

```
[13]: n = 10 # You can adjust this to any value for nxn matrix

target_phase_matrix = generate_random_phase_matrix(n)
target_spin_matrix = binary_phase_to_spin(target_phase_matrix)

detected_phase_matrix = generate_random_phase_matrix(n)
detected_spin_matrix = binary_phase_to_spin(detected_phase_matrix)

[14]: print("Target Phase Matrix:\n")
print_phase_matrix_in_pi(target_phase_matrix)
print("Target Spin Matrix:")
print(target_spin_matrix)
print(f"The shape of the target phase matrix is {target_phase_matrix.shape}.")
print(f"The size of the target phase matrix is {target_phase_matrix.size}.")

print("Detected Random Phase Matrix:")
print_phase_matrix_in_pi(detected_phase_matrix)
print("Detected Random Spin Matrix:")
print(detected_spin_matrix)
print(f"The shape of the random spin matrix is {detected_spin_matrix.shape}.")
print(f"The size of the random spin matrix is {detected_spin_matrix.size}.")

# it determines the the number of pixels or elements in the spatial domain.
```

Target Phase Matrix:

```

    0    0    0 0 0
0      0
0 0    0    0
    0 0 0 0    0
0 0    0    0 0
0 0    0 0 0    0 0
0 0 0      0
0 0    0 0 0 0 0 0
      0 0 0 0    0 0
0 0 0 0 0 0 0 0 0 0
```

Target Spin Matrix:

```
[[-1 -1  1 -1 -1  1 -1  1  1  1]
 [ 1 -1 -1 -1 -1 -1 -1  1 -1 -1]
 [ 1 -1  1 -1 -1  1 -1 -1  1 -1]
 [-1  1  1  1  1 -1 -1  1 -1 -1]
 [ 1  1 -1 -1  1 -1 -1  1 -1  1]
 [ 1  1 -1  1  1  1 -1 -1  1  1]
 [ 1 -1  1 -1  1 -1 -1 -1 -1  1]
 [ 1  1 -1 -1  1  1  1  1  1  1]
 [-1 -1 -1  1  1  1  1 -1  1  1]
```

```
[ 1  1  1  1  1  1  1  1  1  1]
```

The shape of the target phase matrix is (10, 10).
The size of the target phase matrix is 100.

Detected Random Phase Matrix:

```
0
0  0      0  0
  0      0  0  0  0
0  0  0      0
      0  0  0  0  0  0
0  0  0  0  0      0
0      0  0
0      0      0  0  0
  0      0      0  0  0
0  0  0  0  0  0
```

Detected Random Spin Matrix:

```
[[-1  1 -1 -1 -1 -1 -1 -1 -1 -1]
 [ 1 -1  1 -1 -1 -1 -1  1  1 -1]
 [-1  1 -1 -1 -1  1  1  1  1 -1]
 [ 1  1  1 -1 -1 -1  1 -1 -1 -1]
 [-1 -1  1  1  1  1  1 -1  1 -1]
 [ 1 -1  1  1  1  1 -1 -1 -1  1]
 [ 1 -1 -1  1 -1  1 -1 -1 -1 -1]
 [ 1 -1 -1  1 -1 -1  1  1 -1  1]
 [-1  1 -1 -1  1 -1 -1  1  1  1]
 [-1  1  1 -1  1 -1  1  1  1 -1]]
```

The shape of the random spin matrix is (10, 10).
The size of the random spin matrix is 100.

```
[15]: # Create the pixel-like image
# Create the plot
fig, axes = plt.subplots(1, 2, figsize=(10, 5)) # gridspec_kw={'width_ratios': [
    ↪ [1, 1.1]}

# Plot the spin matrix on the left
axes[0].imshow(target_phase_matrix, cmap='viridis')
axes[0].set_title("Pixel Representation of \n Target Phase Matrix")
axes[0].set_xlabel("Pixel Position x")
axes[0].set_ylabel("Pixel Position y ")

# Plot the random phase matrix on the right
cax = axes[1].imshow(detected_phase_matrix, cmap='viridis')
axes[1].set_title("Pixel Representation of \n Detected Phase Matrix")
axes[1].set_xlabel("Pixel Position x")
axes[1].set_ylabel("Pixel Position y")

# Add colorbar for the phase matrix plot
cbar1 = fig.colorbar(cax, ax=axes[0])
```

```

cbar2 = fig.colorbar(cax, ax=axes[1])

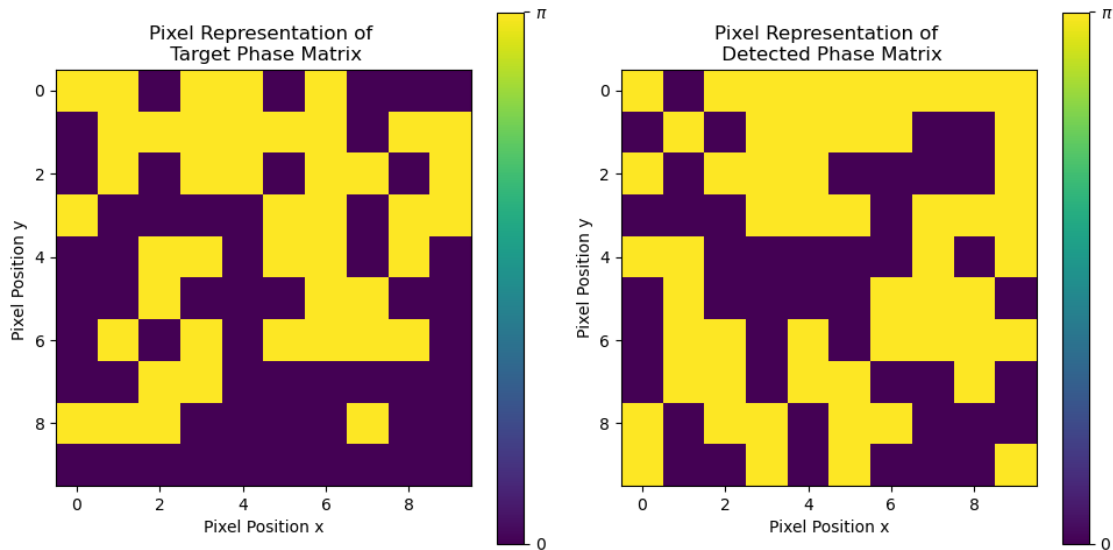
# Set the colorbar ticks and labels
cbar_ticks = [0, np.pi]
cbar_labels = ['0', '$\pi$']

cbar1.set_ticks(cbar_ticks)
cbar1.set_ticklabels(cbar_labels)

cbar2.set_ticks(cbar_ticks)
cbar2.set_ticklabels(cbar_labels)

# Show the plots
plt.tight_layout()
plt.show()

```



```

[16]: print(f"Target phase matrix is \n{target_phase_matrix}.\n")
      print(f"Detected phase matrix is \n {detected_phase_matrix}.")

```

Target phase matrix is

```

[[3.14159265 3.14159265 0.          3.14159265 3.14159265 0.
  3.14159265 0.          0.          0.          ]
 [0.          3.14159265 3.14159265 3.14159265 3.14159265 3.14159265
  3.14159265 0.          3.14159265 3.14159265]
 [0.          3.14159265 0.          3.14159265 3.14159265 0.
  3.14159265 3.14159265 0.          3.14159265]
 [3.14159265 0.          0.          0.          0.          3.14159265
  3.14159265 0.          3.14159265 3.14159265]
 [0.          0.          3.14159265 3.14159265 0.          3.14159265

```

```

3.14159265 0.          3.14159265 0.          ]
[0.          0.          3.14159265 0.          0.          0.
3.14159265 3.14159265 0.          0.          ]
[0.          3.14159265 0.          3.14159265 0.          3.14159265
3.14159265 3.14159265 3.14159265 0.          ]
[0.          0.          3.14159265 3.14159265 0.          0.
0.          0.          0.          0.          ]
[3.14159265 3.14159265 3.14159265 0.          0.          0.
0.          3.14159265 0.          0.          ]
[0.          0.          0.          0.          0.          0.
0.          0.          0.          0.          ]].

```

Detected phase matrix is

```

[[3.14159265 0.          3.14159265 3.14159265 3.14159265 3.14159265
3.14159265 3.14159265 3.14159265 3.14159265]
[0.          3.14159265 0.          3.14159265 3.14159265 3.14159265
3.14159265 0.          0.          3.14159265]
[3.14159265 0.          3.14159265 3.14159265 3.14159265 0.
0.          0.          0.          3.14159265]
[0.          0.          0.          3.14159265 3.14159265 3.14159265
0.          3.14159265 3.14159265 3.14159265]
[3.14159265 3.14159265 0.          0.          0.          0.
0.          3.14159265 0.          3.14159265]
[0.          3.14159265 0.          0.          0.          0.
3.14159265 3.14159265 3.14159265 0.          ]
[0.          3.14159265 3.14159265 0.          3.14159265 0.
3.14159265 3.14159265 3.14159265 3.14159265]
[0.          3.14159265 3.14159265 0.          3.14159265 3.14159265
0.          0.          3.14159265 0.          ]
[3.14159265 0.          3.14159265 3.14159265 0.          3.14159265
3.14159265 0.          0.          0.          ]
[3.14159265 0.          0.          3.14159265 0.          3.14159265
0.          0.          0.          3.14159265]].

```

The above code only involves the phase values (0 and π) mapping to the spin variables, but this mapping effectively incorporates the exponential term.

$$\sigma_j = e^{i\phi_j} \quad (1)$$

It means that phase $\phi_j = 0$ corresponds to $\sigma_j e^{i \times 0} = 1$, phase $\phi_j = \pi$ corresponds to $\sigma_j e^{i\pi} = -1$. This completely represent the phases of the spin variables that maps the phases values to the spin variables: $0 \rightarrow 1$ and $\pi \rightarrow -1$.

1.3.1 The experimental setup:

1. Continuous Laser source with wavelength $\lambda = 532nm$.
2. Holoeye LC-R 720
3. resolution 1280×768 pixels
4. Pixel pitch $20 \times 20\mu m$

5. Active area to host $N = L \times L$
6. foocal lens $f = 500mm$
7. Intensity is measure on $M = 18 \times 18$ spatial modes obtained grouping 16×16 camera pixels

Pixel pitch refers to the distance between the centers of adjacent pixels. If the pixels are square, this values represents both the width and the height of each pixel. From the given information on the paper, the pixel pitch of the SLM is $20 \times 20\mu m$, which indicate the each pixel is a square with a side length of $20\mu m$.

In the spatial domain, a rectangular function represents a finite-width window. The function can be described as follow: If the the domain is within the range of $x \in [-W, W]$ then it is 1, otherwise it is 0. This function models the finite size of a pixel because it is nonzero only within the pixel's extent and zero outside it.

In the frequency domain, the Fourier transform of the rectangular function in the spatial domain is a sinc function. The sinc function describes how the spatial domain's retangular function (the pixel's finite size) affects the ditribution of spatial frequencies.

W refers to the width of the rectangular function in the spatial domain. When the rectangular function is taken the Fourier transform, a sinc function in the frequency domain is obtained.

The normalized $\tilde{\delta}_W$ models the pixel of finite size $2W$, so that

$$k_j = 2Wj, \quad j = 1, 2, 3, \dots, n. \quad (2)$$

$2W$ represents the width of the pixel in the spatial domain. Since the pixel pitch is $20\mu m$, the pixel width $2W$ would naturally be $20\mu m$, assuming the pixels are fully utilized.

We define `j_values` as a range from 1 to n.

```
j_values = np.arange(1, n+1)
```

```
k_j = 2 * W * j
```

The normalized sinc function is expressed as:

$$\text{sinc}x = \frac{\sin \pi x}{\pi x} \quad (3)$$

The sinc function is centered at $k_j = 2Wj$, which means that its shape and position vary with the index j . This reflects the fact that each pixel contributes a different spatial frequency component to the overall electric field. The sinc function is introduced as expressed as follow:

$$\tilde{\delta}_W(k - k_j) = \frac{\sin W(k - k_j)}{W(k - k_j)}, \quad j = 1, 2, 3, \dots, n \quad (4)$$

The sinc function $\sin W(k - k_j)$ models the pixel's finite size and its response in the spatial frequency domain. $W(k - k_j)$ determine the width of the sinc function, and hence the spatial localization of the electric field in the frequency domain.

he larger the W (indicating a larger pixel in the spatial domain), the narrower the sinc function in the spatial frequency domain. This indicates that the electric field is more concentrated in the

spatial frequency domain, leading to a broader (less localized) distribution in the spatial domain. Conversely, a smaller W results in a broader sinc function in the spatial frequency domain, which corresponds to a more localized (sharper) distribution in the spatial domain. $\tilde{\delta}_W(k - k_j)$ represents the Fourier transform of a rectangular sinc function, which corresponds to the finite size of the pixel in the spatial domain by describing how electric field being modulated by the finite size of the pixel in the Fourier domain. $k - k_j$ represents the spatial frequency difference between a particular frequency k and the frequency k_j associated with the pixel position.

```
delta_w_k = np.sinc(W * (k - k_j))
```

The k array represents the spatial frequency grid. Its length determines the number of frequency components considered. The specific values in the k array depend on the pixel pitch, the wavelength, and the desired spatial frequency range. The pixel pitch and wavelength of the light determine the spatial frequency bandwidth. The desired spatial resolution in the Fourier domain will influence the range of k values.

To avoid aliasing, the maximum frequency in k should be at least twice the Nyquist frequency, which is determined by the pixel pitch and the wavelength. The Nyquist frequency, which represents the maximum resolvable spatial frequency.

1.3.2 2. Light propagation:

The function `calculate_spatial_frequency_grid` calculates the spatial frequency grid based on the pixel pitch, wavelength, and desired number of points. The idea is to create a grid that represents the spatial frequencies based on the Nyquist frequency and the specified factor. The `factor` parameter is to allow adjusting the range of k based on the specific requirements. A larger factor will result in a wider range of spatial frequencies being considered. The `num_points` parameter can be adjusted to control the resolution in the Fourier domain. The Nyquist frequency `nyquist_frequency` is a crucial factor in determining the appropriate range of k values.

k should match the dimensions of the spatial domain grid corresponding to the phase matrix. A grid of spatial frequencies k_x and k_y is created using `np.meshgrid`.

““Calculates the spatial frequency grid. Args: pixel_pitch: The pixel pitch. wavelength: The wavelength of the light. num_points: The number of points sampled in the spatial frequency domain, which is used to determined the resolution of the spatial frequency grid. The larger value means finer resolution. factor: A factor to determine the maximum frequency Returns: The spatial frequency grid as a NumPy array. Nyquist frequency: It represents half of the maximum frequency in the spatial frequency grid to avoid aliasing, which is determined by the pixel pitch and the wavelength.”“ ”

```
[17]: def calculate_spatial_frequency_grid(pixel_pitch, wavelength, num_points,
      ↪ factor = 1):
      """Calculates the spatial frequency grid.

      Args:
      pixel_pitch: The pixel pitch.
      wavelength: The wavelength of the light.
      num_points: The number of points sampled in the spatial frequency
      ↪ domain,
```

which is used to determine the resolution of the spatial frequency grid.

The larger value means finer resolution.

factor: A factor to determine the maximum frequency (default: 2).

Returns:

The spatial frequency grid as a NumPy array.

Nyquist frequency:

It represents half of the maximum frequency in the spatial frequency grid to avoid aliasing,

which is determined by the pixel pitch and the wavelength.

"""

Calculate the Nyquist frequency

*nyquist_frequency = 1 / (2 * pixel_pitch)*

Determine the maximum frequency based on the factor

*max_frequency = factor * nyquist_frequency*

Create the spatial frequency grid

k = np.linspace(-max_frequency, max_frequency, num_points)

kx, ky = np.meshgrid(k, k) # Create a grid of kx, ky

return kx, ky

`num_points` represents the number of spatial samples or grid points in each dimension of the simulation, which corresponds to the size of `target_spin_matrix`. The number of points is set by the number of rows or columns (in our case it is a square matrix) in the `target_spin_matrix`. The matrix represents the form of pixelized spin system, where each entry corresponds to the spin variable at a given point on the grid. The resolution of the simulation (number of points) should match the desired accuracy and the spatial extent of the system.

```
[18]: xi = 1.0 # Scalar amplitude coefficient
pixel_pitch = 20e-6 # Pixel pitch in meters
W = pixel_pitch / 2 # Half of Pixel pitch
wavelength = 532e-9 # Wavelength in meters
num_points = target_spin_matrix.shape[0]
k = calculate_spatial_frequency_grid(pixel_pitch, wavelength, 100)
```

```
[19]: # print(k)
# print(k.shape)
# print(k.size)
```

The modulated light propagates through an optical system. Interactions between different light paths create interference patterns. When two or more light waves overlap, they interfere with each other. The resulting intensity depends on the phase difference between the waves. This principle is fundamental to holography, interferometry, and optical computing.

The wave we are considering in the Fourier plane is the Fourier transform of the beam after the

spatial light modulation. This is a function that has its shape, where there is the spin corresponds to the local phase, and ξ is the local amplitude

$$\tilde{E}(k) = \sum_j \xi_j \sigma_j \tilde{\delta}_W(k - k_j) \quad (5)$$

A higher ξ would lead to a higher intensity, as intensity is proportional to the square of the electric field. ξ can be normalized depending on the total power or energy in the system. Without knowing the exact value of ξ , setting it to 1 is reasonable as normalization.

(The specialized modulator is typically located at a position which is fully conjugated with respect to the camera. It means that by using the law of Optics, what we detect at the camera is the Fourier transform of the beam after spatial light modulator.)

The light propagation through the optical system is designed to mimic the interactions between spins in the Ising model. The interference patterns that form represent the energy of the system.

$$\begin{aligned} E(x) &= \sum_j \xi_j \sigma_j \delta_W(x - x_j) e^{2ik_j x} \\ &= \delta_W(x - x_j) \sum_j \xi_j \sigma_j e^{2iWjx} \end{aligned} \quad (6)$$

```
E_x = np.fft.ifft(E_k)
```

$$\begin{aligned} I(x) &= |E(x)|^2 \\ &= E^*(x) E(x) \\ &= \sum_j \xi_j^* \sigma_j^* \delta_W^*(x - x_j) e^{-2iWjx} \sum_h \xi_h \sigma_h \delta_W(x - x_h) e^{2iWhx} \\ &= \sum_j \sum_h \xi_j \xi_h \sigma_j \sigma_h \delta_W^*(x - x_j) \delta_W(x - x_h) e^{2iW(h-j)x} \\ &= \sum_{jh} \xi_j \xi_h \sigma_j \sigma_h \delta_W^2(x) e^{2iW(h-j)x} \end{aligned} \quad (7)$$

```
I_x = np.abs(E_x)**2
```

Below is the overall coding for calculating the Intensity from combining all the spatial frequency grid in the frequency domain and then do the fourier transform. “ “ “Calculates the intensity I(x) based on the given parameters in the Fourier domain.

Args:

xi: A scalar representing the amplitude coefficient.
spin_matrix: A numpy array representing the spin variables.
W: The pixel pitch.
kx, ky: The spatial frequency coordinates.

Returns:

The calculated intensity I(x).

```
"""
```

```
[20]: def calculate_intensity_fourier(xi, spin_matrix, W, kx, ky):
    """Calculates the intensity I(x) based on the given parameters in the
    ↪Fourier domain.

    Args:
        xi: A scalar representing the amplitude coefficient.
        spin_matrix: A numpy array representing the spin variables.
        W: The pixel pitch.
        kx, ky: The spatial frequency coordinates.

    Returns:
        The calculated intensity I(x).
    """

    # Determine n from the input directly
    n = spin_matrix.size
    # Initialize E_k in the Fourier domain (complex field)
    E_k = np.zeros_like(kx, dtype=complex)

    # Double summation over j and h
    for j in range(n):
        row_j, col_j = divmod(j, spin_matrix.shape[1])
        k_jx = 2 * W * col_j
        k_jy = 2 * W * row_j

        for h in range(n):
            row_h, col_h = divmod(h, spin_matrix.shape[1])
            k_hx = 2 * W * col_h
            k_hy = 2 * W * row_h

            # Calculate  $\delta W^2(x)$  using the sinc function for both j and h
            delta_w_square_j = np.sinc(W * (kx - k_jx)) * np.sinc(W * (ky -
            ↪k_jy))
            delta_w_square_h = np.sinc(W * (kx - k_hx)) * np.sinc(W * (ky -
            ↪k_hy))

            # Phase factor  $\exp(2iW(h-j)x)$ 
            phase_factor = np.exp(2j * W * ((col_h - col_j) * kx + (row_h -
            ↪row_j) * ky))

            # Accumulate contributions to the electric field E_k
            E_k += (xi**2 * spin_matrix[row_j, col_j].conjugate() *
            ↪spin_matrix[row_h, col_h] *
                        delta_w_square_j * delta_w_square_h.conjugate() *
            ↪phase_factor)

    # Inverse Fourier Transform to return to the spatial domain
```

```

E_x = np.fft.ifft2(E_k)

# Intensity I(x) is the square of the magnitude of the electric field
I_x = np.abs(E_x) ** 2
return I_x

```

The function computes the intensity based on the 2D grid of spatial frequencies k_x and k_y . The summation process accounts for both the parameters to ensure the pattern is calculated correctly. The phase factor corresponds to the phase difference between the j -th and h -th pixels. $\text{delta_w_k} = \text{np.sinc}(W * (k_x - k_{jx})) * \text{np.sinc}(W * (k_y - k_{jy}))$ uses the normalized sinc function to model the pixel's response in the spatial frequency domain.

```

[21]: # Record the start time
start_time = time.time()

xi = 1.0 # Scalar amplitude coefficient
pixel_pitch = 20e-6 # Pixel pitch in meters
W = pixel_pitch / 2 # Half of Pixel pitch
wavelength = 532e-9 # Wavelength in meters
num_points = target_spin_matrix.shape[0]

# Calculate spatial frequency grid
kx, ky = calculate_spatial_frequency_grid(pixel_pitch, wavelength, num_points)

# Calculate intensity pattern
I_x_Target = calculate_intensity_fourier(xi, target_spin_matrix, W, kx, ky)
I_x_Detected = calculate_intensity_fourier(xi, detected_spin_matrix, W, kx, ky)

# Record the end time
end_time = time.time()

# Calculate elapsed time
elapsed_time = end_time - start_time

# Print the elapsed time
print(f"Computation Time: {elapsed_time:.4f} seconds")

```

Computation Time: 0.5043 seconds

```

[22]: I_x_Target

```

```

[22]: array([[4.16076318e+04, 5.86602080e+03, 2.39696186e+01, 1.84939865e-01,
           5.86626219e-02, 9.98923042e-17, 5.86626219e-02, 1.84939865e-01,
           2.39696186e+01, 5.86602080e+03],
          [3.34800421e+03, 8.93422630e+01, 3.98498411e+01, 2.07940563e-01,
           7.87518405e-02, 4.05536655e-01, 9.57636384e-01, 2.19838287e+00,
           1.77690462e+00, 1.12778276e+03],
          [3.88414886e+02, 1.15420733e+03, 8.51574369e-01, 1.47440120e+00,

```

```

1.37915924e+00, 1.27617824e+00, 1.30935318e+00, 1.40874309e+00,
2.77053858e-02, 1.28047987e+02],
[2.05408076e+01, 1.77913931e+02, 4.61647493e-01, 1.91619089e-01,
2.03861757e-01, 1.96015312e-01, 2.00971581e-01, 1.95932878e-01,
1.32974138e-01, 6.24119109e+01],
[2.44648200e+00, 8.86276469e+01, 2.87338657e-01, 1.05621474e-01,
1.15590505e-01, 1.12339731e-01, 1.15028302e-01, 1.07284321e-01,
1.78059231e-01, 5.65694147e+01],
[2.28971621e-15, 6.30053275e+01, 2.13906957e-01, 9.02940861e-02,
9.86958718e-02, 9.62969625e-02, 9.86958718e-02, 9.02940861e-02,
2.13906957e-01, 6.30053275e+01],
[2.44648200e+00, 5.65694147e+01, 1.78059231e-01, 1.07284321e-01,
1.15028302e-01, 1.12339731e-01, 1.15590505e-01, 1.05621474e-01,
2.87338657e-01, 8.86276469e+01],
[2.05408076e+01, 6.24119109e+01, 1.32974138e-01, 1.95932878e-01,
2.00971581e-01, 1.96015312e-01, 2.03861757e-01, 1.91619089e-01,
4.61647493e-01, 1.77913931e+02],
[3.88414886e+02, 1.28047987e+02, 2.77053858e-02, 1.40874309e+00,
1.30935318e+00, 1.27617824e+00, 1.37915924e+00, 1.47440120e+00,
8.51574369e-01, 1.15420733e+03],
[3.34800421e+03, 1.12778276e+03, 1.77690462e+00, 2.19838287e+00,
9.57636384e-01, 4.05536655e-01, 7.87518405e-02, 2.07940563e-01,
3.98498411e+01, 8.93422630e+01]])

```

```

[23]: # Print the intensity patterns
print(f"Target Intensity Pattern:\n {I_x_Target}\n \n")
print(f"Detected Intensity Pattern:\n {I_x_Detected}\n \n")
print(f"kx shape: {kx.shape}\n")
print(f"ky shape: {ky.shape}\n")
print(f"Number of points: {num_points}\n")

```

Target Intensity Pattern:

```

[[4.16076318e+04 5.86602080e+03 2.39696186e+01 1.84939865e-01
 5.86626219e-02 9.98923042e-17 5.86626219e-02 1.84939865e-01
 2.39696186e+01 5.86602080e+03]
[3.34800421e+03 8.93422630e+01 3.98498411e+01 2.07940563e-01
 7.87518405e-02 4.05536655e-01 9.57636384e-01 2.19838287e+00
 1.77690462e+00 1.12778276e+03]
[3.88414886e+02 1.15420733e+03 8.51574369e-01 1.47440120e+00
 1.37915924e+00 1.27617824e+00 1.30935318e+00 1.40874309e+00
 2.77053858e-02 1.28047987e+02]
[2.05408076e+01 1.77913931e+02 4.61647493e-01 1.91619089e-01
 2.03861757e-01 1.96015312e-01 2.00971581e-01 1.95932878e-01
 1.32974138e-01 6.24119109e+01]
[2.44648200e+00 8.86276469e+01 2.87338657e-01 1.05621474e-01
 1.15590505e-01 1.12339731e-01 1.15028302e-01 1.07284321e-01
 1.78059231e-01 5.65694147e+01]
[2.28971621e-15 6.30053275e+01 2.13906957e-01 9.02940861e-02

```

9.86958718e-02 9.62969625e-02 9.86958718e-02 9.02940861e-02
 2.13906957e-01 6.30053275e+01]
 [2.44648200e+00 5.65694147e+01 1.78059231e-01 1.07284321e-01
 1.15028302e-01 1.12339731e-01 1.15590505e-01 1.05621474e-01
 2.87338657e-01 8.86276469e+01]
 [2.05408076e+01 6.24119109e+01 1.32974138e-01 1.95932878e-01
 2.00971581e-01 1.96015312e-01 2.03861757e-01 1.91619089e-01
 4.61647493e-01 1.77913931e+02]
 [3.88414886e+02 1.28047987e+02 2.77053858e-02 1.40874309e+00
 1.30935318e+00 1.27617824e+00 1.37915924e+00 1.47440120e+00
 8.51574369e-01 1.15420733e+03]
 [3.34800421e+03 1.12778276e+03 1.77690462e+00 2.19838287e+00
 9.57636384e-01 4.05536655e-01 7.87518405e-02 2.07940563e-01
 3.98498411e+01 8.93422630e+01]]

Detected Intensity Pattern:

[[4.79941173e+03 3.23653952e+02 7.00523808e+00 5.09307601e-01
 6.77537349e-02 9.81099767e-18 6.77537349e-02 5.09307601e-01
 7.00523808e+00 3.23653952e+02]
 [7.25221703e+00 2.47877414e+02 2.68666938e-01 1.13507899e+00
 9.28843067e-01 8.91616828e-01 1.04341862e+00 1.58491046e+00
 3.37955763e+00 3.63986948e+01]
 [2.66845191e+01 1.67989561e+02 2.04018643e+00 1.40643681e+00
 9.14474373e-01 7.48421988e-01 7.40710126e-01 8.63069234e-01
 2.31617454e-01 5.76831258e+01]
 [2.06408697e-01 2.93103685e+01 1.79366953e-01 2.08670656e-01
 1.45983379e-01 1.26588712e-01 1.33195247e-01 1.69020130e-01
 6.73385763e-02 1.81750255e+01]
 [1.80391103e-02 1.59044511e+01 7.29346315e-02 1.11779613e-01
 8.08707289e-02 7.19192717e-02 7.75988139e-02 1.01661866e-01
 4.61136658e-02 1.28370103e+01]
 [8.08824305e-21 1.24738510e+01 4.86476604e-02 9.10922300e-02
 6.77092776e-02 6.14721430e-02 6.77092776e-02 9.10922300e-02
 4.86476604e-02 1.24738510e+01]
 [1.80391103e-02 1.28370103e+01 4.61136658e-02 1.01661866e-01
 7.75988139e-02 7.19192717e-02 8.08707289e-02 1.11779613e-01
 7.29346315e-02 1.59044511e+01]
 [2.06408697e-01 1.81750255e+01 6.73385763e-02 1.69020130e-01
 1.33195247e-01 1.26588712e-01 1.45983379e-01 2.08670656e-01
 1.79366953e-01 2.93103685e+01]
 [2.66845191e+01 5.76831258e+01 2.31617454e-01 8.63069234e-01
 7.40710126e-01 7.48421988e-01 9.14474373e-01 1.40643681e+00
 2.04018643e+00 1.67989561e+02]
 [7.25221703e+00 3.63986948e+01 3.37955763e+00 1.58491046e+00
 1.04341862e+00 8.91616828e-01 9.28843067e-01 1.13507899e+00
 2.68666938e-01 2.47877414e+02]]

kx shape: (10, 10)

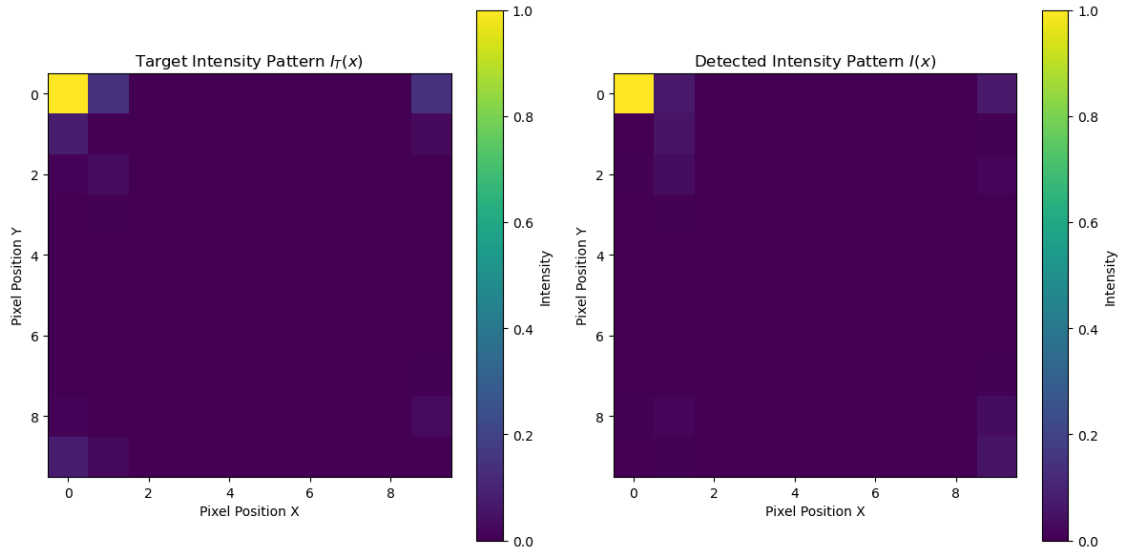
ky shape: (10, 10)

Number of points: 10

Both the target and the detected intensity are normalized before running the optimization to match the shape of both patterns.

```
[24]: I_x_Target_normalized = (I_x_Target - np.min(I_x_Target)) / (np.max(I_x_Target) -  
    ↪ np.min(I_x_Target))  
I_x_Detected_normalized = (I_x_Detected - np.min(I_x_Detected)) / (np.  
    ↪ max(I_x_Detected) - np.min(I_x_Detected))
```

```
[25]: # Create a figure with two subplots side by side  
fig, axes = plt.subplots(1, 2, figsize=(12, 6))  
  
# Plot the first intensity pattern on the left  
im1 = axes[0].imshow(I_x_Target_normalized, cmap='viridis')  
axes[0].set_title("Target Intensity Pattern  $I_T(x)$ ")  
axes[0].set_xlabel("Pixel Position X")  
axes[0].set_ylabel("Pixel Position Y")  
fig.colorbar(im1, ax=axes[0], label="Intensity")  
  
# Plot the second intensity pattern on the right  
im2 = axes[1].imshow(I_x_Detected_normalized, cmap='viridis')  
axes[1].set_title("Detected Intensity Pattern  $I(x)$ ")  
axes[1].set_xlabel("Pixel Position X")  
axes[1].set_ylabel("Pixel Position Y")  
fig.colorbar(im2, ax=axes[1], label="Intensity")  
  
# Adjust layout to prevent overlap  
plt.tight_layout()  
  
# Display the plots  
plt.show()
```

From the figure above, it shows that the zero-frequency component is dominating the intensity pattern and is concentrated in the upper-left corner of the output image. This happens because the zero frequency is located at the (0,0) index of the Fourier-transformed array. To visualize the pattern properly, using `np.fft.fftshift` can shift the zero-frequency component of the 2D Fourier transform to the centre of the frequency domain image. This allows for observing any higher-frequency components or structures in the pattern.

```
[26]: # Apply fftshift to center the zero-frequency component
I_x_Target_shifted = np.fft.fftshift(I_x_Target_normalized)
I_x_Detected_shifted = np.fft.fftshift(I_x_Detected_normalized)

# Create a figure with two subplots side by side
fig, axes = plt.subplots(1, 2, figsize=(12, 6))

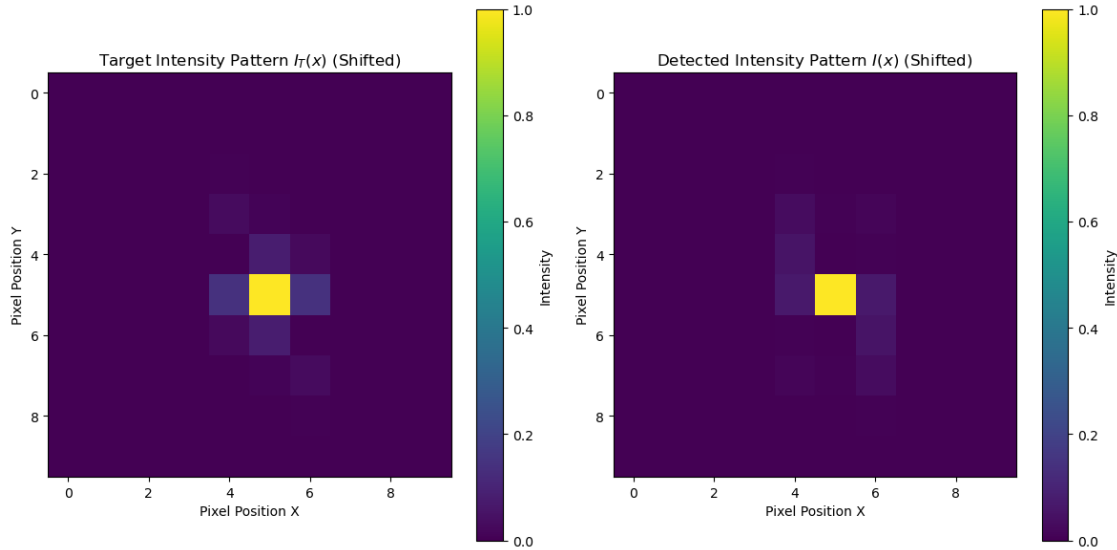
# Plot the first intensity pattern on the left
im1 = axes[0].imshow(I_x_Target_shifted, cmap='viridis')
axes[0].set_title("Target Intensity Pattern  $I_T(x)$  (Shifted)")
axes[0].set_xlabel("Pixel Position X")
axes[0].set_ylabel("Pixel Position Y")
fig.colorbar(im1, ax=axes[0], label="Intensity")

# Plot the second intensity pattern on the right
im2 = axes[1].imshow(I_x_Detected_shifted, cmap='viridis')
axes[1].set_title("Detected Intensity Pattern  $I(x)$  (Shifted)")
axes[1].set_xlabel("Pixel Position X")
axes[1].set_ylabel("Pixel Position Y")
fig.colorbar(im2, ax=axes[1], label="Intensity")

# Adjust layout to prevent overlap
```

```
plt.tight_layout()

# Display the plots
plt.show()
```



```
[27]: # Calculate and print the min and max values of intensity patterns
target_min = np.min(I_x_Target_shifted)
target_mean = np.mean(I_x_Target_shifted)
target_max = np.max(I_x_Target_shifted)
detected_min = np.min(I_x_Detected_shifted)
detected_mean = np.mean(I_x_Detected_shifted)
detected_max = np.max(I_x_Detected_shifted)

# Prepare markdown string
markdown_string = f"""
**Target Intensity Min Value:** {target_min:.16e}
**Target Intensity Mean Value:** {target_mean:.4f}
**Target Intensity Max Value:** {target_max:.4f}

**Detected Intensity Min Value:** {detected_min:.16e}
**Detected Intensity Mean Value:** {detected_mean:.4f}
**Detected Intensity Max Value:** {detected_max:.4f}

The values in the target intensity range from **{target_min:.16e}** to
↳ **{target_max:.4f}** with a mean value of **{target_mean:.4f}** which
↳ suggests a strong peak in intensity at a particular point (a focused spot or
↳ central region) and much smaller intensity values elsewhere.
```

```
This is consistent with an optical setup that tries to focus light into a small_
↪region in the far-field. The detected pattern has minimum values of_
↪**{detected_min:.16e}** to **{detected_max:.4f}** with a mean value of_
↪**{detected_mean:.4f}**.
```

```
The overall structure is roughly comparable to the target intensity pattern,_
↪with a peak and some distribution around it. Both the target and detected_
↪intensity patterns exhibit some symmetry, which is typical of far-field_
↪diffraction patterns where the intensity is spread symmetrically around a_
↪central peak.
```

```
The presence of low-intensity values in specific areas, especially near-zero_
↪points, is also consistent with diffraction effects (values close to 1 due_
↪to constructive interference, and zeros due to destructive interference).
```

```
The phase relationship between different parts of the wavefront significantly_
↪affects the final intensity distribution, and the slight mismatch between_
↪target and detected intensities could stem from incomplete optimization of_
↪the phases. Small differences in phase can result in larger discrepancies in_
↪intensity due to interference effects.
```

```
"""
```

```
[28]: display(Markdown(markdown_string))
```

Target Intensity Min Value: 0.0000000000000000e+00

Target Intensity Mean Value: 0.0161

Target Intensity Max Value: 1.0000

Detected Intensity Min Value: 0.0000000000000000e+00

Detected Intensity Mean Value: 0.0141

Detected Intensity Max Value: 1.0000

The values in the target intensity range from **0.0000000000000000e+00** to **1.0000** with a mean value of **0.0161** which suggests a strong peak in intensity at a particular point (a focused spot or central region) and much smaller intensity values elsewhere.

This is consistent with an optical setup that tries to focus light into a small region in the far-field. The detected pattern has minimum values of **0.0000000000000000e+00** to **1.0000** with a mean value of **0.0141**.

The overall structure is roughly comparable to the target intensity pattern, with a peak and some distribution around it. Both the target and detected intensity patterns exhibit some symmetry, which is typical of far-field diffraction patterns where the intensity is spread symmetrically around a central peak.

The presence of low-intensity values in specific areas, especially near-zero points, is also consistent with diffraction effects (values close to 1 due to constructive interference, and zeros due to destructive interference).

The phase relationship between different parts of the wavefront significantly affects the final inten-

sity distribution, and the slight mismatch between target and detected intensities could stem from incomplete optimization of the phases. Small differences in phase can result in larger discrepancies in intensity due to interference effects.

3. Energy calculation:

The goal is to find the phase configuration that minimizes this energy, which corresponds to the ground state of the Ising Hamiltonian. The intensity of the interference pattern is related to the energy of the Ising system.

$$\text{Energy} = \int [I(x)]^2 dx \quad (8)$$

The cost function is defined as the difference of the intensity, which corresponds to finding the ground state of the Ising Hamiltonian.

$$\text{Cost Function} = f = ||I(x) - I_T(x)|| \quad (9)$$

This function measures how far the current spin configuration is from the target intensity.

"""

Calculates the energy (or cost) based on the difference between detected intensity and target intensity.

Args:

I_detected: The detected intensity pattern.

I_target: The target intensity pattern.

Returns:

The energy (cost) value.

"""

```
[29]: # Calculate the detected intensity pattern using the Fourier-based function
I_detected_shifted = np.fft.fftshift(calculate_intensity_fourier(xi,
    ↪ detected_spin_matrix, W, kx, ky))

# Calculate the target intensity pattern using the Fourier-based function
I_target_shifted = np.fft.fftshift(calculate_intensity_fourier(xi,
    ↪ target_spin_matrix, W, kx, ky))

@tf.function
# compute the energy difference (cost function)
# def calculate_energy(I_detected_shifted, I_target_shifted):

#     return np.sum((I_target_shifted - I_detected_shifted)**2)

def calculate_energy(I_x_Detected_normalized, I_x_Target_normalized):
    """Calculate energy difference between detected and target intensity."""
```

```

    return tf.reduce_sum(tf.square(I_x_Detected_normalized -
↪I_x_Target_normalized))

# # Calculate the energy difference
# energy_difference = calculate_energy(I_detected_shifted, I_target_shifted)

# # Print the energy difference
# print(f"Energy Difference: {energy_difference}")

```

1.4 Optimization: Monte Carlo Method

The ground state energy can be found using Monte Carlo method by establishing the connection between the spin configurations and the associated energy levels. The ground state energy corresponds to the minimum energy configuration reached during simulation processes. This represents the spin configuration, or phase matrix, that produces the closet match to the target intensity pattern.

The order of magnitude between the two patterns are large. It is advisable to implement Monto-Carlo method optimization for the evolution of the detected system to match the target intensity distribution such that the detect pattern grow closer to the target over more iterations.

The Monte Carlo method is essential to simulate the spin evolution, which is based on the concept of flipping spin or clusers of spins and measuring whether this change leads to a decrease in Hamiltonian, i.e., the differece bwtween the target intensity and the detected intensity.

Initially, a cluster of spins (a few pixels on the SLM) would be randomly flipped and the resulting intensity pattern would be calculated. After each cluster flip, the system iteratively adjusts the phase modulations such that the resulting intensity pattern $I(x)$ would match the arbitrary target intensity pattern $I_T(x)$ (the difference between the new intensity pattern and the target intensity, i.e. $f = |I(x) - I_T(x)|$) to minimize the energy.

If the new configuration brings the system closer to the target, i.e., the difference is reduced, the new configuration would be retained. The process will be repeated for multiple iterations until the cost function is minimized, i.e. no furtehr configuration improves the energy significantly.

Since the system's evolution is purely driven by minimizing the cost function f , the Hamiltonian $H = -\sum_{jh} J_{jh} \sigma_j \sigma_h$ is not explicitly calculated. The spin-spin interaction J_{jh} emerge naturally from the modulation of light on the SLM and the feedback from the far-field intensity pattern.

Energy values `energy_values` is being stored into list at each iteration, and then the energy difference `energy_diffs = [energy_values[i] - energy_values[i-1] for i in range(1, len(energy)values))]` between each iterations is calculated. To check if the energy difference is decreasing, we can plot the energy values at each iteration. We can also visualize the final phase matrix to see if it matches the target phase matrix. If there is a plateau in the plot, it indicates that the energy difference converges. Finally, after the simulation is completed, the plot is delivered to observe how the energy stabilizes over time.

Plotting both the energy differences and energy values over iterations. The plateau behavior is checked to verify if the process has likely converged to the ground state.

1.4.1 Consideration

it is important to gradually increase the cluster sizer to avoid getting stuck in local minima, this approach would be implemented by starting with single spoin flips and gradually increasing the cluster size. In the experiment, random noise and fluctuation would occur, introducing random noise in the intensity calculation is essential to simulate real-world conditions.

```
[30]: # def monte_carlo_ground_state(detected_phase_matrix, I_target_shifted, xi, W,
      ↪kx, ky, iterations = 100):
      #     n = detected_phase_matrix.shape[0]

      #     # Convert the initial phase matrix to a spin matrix
      #     detected_spin_matrix = binary_phase_to_spin(detected_phase_matrix)

      #     # Calculate the initial intensity and energy
      #     I_x_Detected = calculate_intensity_fourier(xi, detected_spin_matrix, W,
      ↪kx, ky)
      #     current_energy = calculate_energy(I_x_Detected, I_target_shifted)

      #     # Track the ground state energy and energy differences
      #     ground_state_energy = current_energy
      #     energy_values = [current_energy] # Store energies

      #     for i in range(iterations):
      #         # Randomly flip a small cluster of spins (phase flip in a small
      ↪region)
      #         new_phase_matrix = detected_phase_matrix.copy()
      #         cluster_size = np.random.randint(1, 5) # Choose a small cluster size
      #         random_indices = np.random.choice(n * n, cluster_size, replace=False)

      #         # Flip the phase at the selected positions
      #         for index in random_indices:
      #             row, col = divmod(index, n)
      #             new_phase_matrix[row, col] = np.pi - new_phase_matrix[row, col]

      #         # Convert the new phase matrix to a spin matrix
      #         new_spin_matrix = binary_phase_to_spin(new_phase_matrix)

      #         # Calculate new intensity and energy
      #         new_intensity = calculate_intensity_fourier(xi, new_spin_matrix, W,
      ↪kx, ky)
      #         new_energy = calculate_energy(new_intensity, I_target_shifted)

      #         # Track energy values
      #         energy_values.append(new_energy)

      #         # Calculate energy difference between consecutive iterations
      #         energy_diff = new_energy - current_energy
```

```

#         # Accept the new configuration if the energy decreases
#         if new_energy < current_energy:
#             final_detected_phase_matrix = new_phase_matrix # Accept the new
↪phase matrix
#             current_energy = new_energy

#         # Update ground state energy if necessary
#         if current_energy < ground_state_energy:
#             ground_state_energy = current_energy

#         # Optional: Print or track the progress
#         if i % 10 == 0:
#             print(f"Iteration {i}, Energy: {current_energy}, Ground State
↪Energy: {ground_state_energy}, Energy Difference: {energy_diff}")

#         # Calculate the energy differences
#         energy_diffs = [abs(energy_values[i] - energy_values[i-1]) for i in
↪range(1, len(energy_values))]

#         # Record the end time
#         end_time = time.time()

#         # Calculate elapsed time
#         elapsed_time = end_time - start_time

#         # Plot energy differences over iterations
#         plt.figure(figsize=(10, 6))

#         # Plot 1: Energy differences
#         plt.subplot(2, 1, 1)
#         plt.plot(energy_diffs, label="Energy Difference", color='b')
#         plt.xlabel("Iteration")
#         plt.ylabel("Energy Difference")
#         plt.title("Energy Difference Between Iterations")
#         plt.grid(True)
#         plt.legend()

#         # Plot 2: Energy values
#         plt.subplot(2, 1, 2)
#         plt.plot(energy_values, label="Energy Values", color='g')
#         plt.xlabel("Iteration")
#         plt.ylabel("Energy")
#         plt.title("Energy Values Over Iterations")
#         plt.grid(True)
#         plt.legend()

```

```

#     plt.tight_layout()
#     plt.show()

#     # Plot the histogram of energy values
#     plt.figure(figsize=(10, 6))
#     plt.hist(energy_values, bins=50, color='purple', alpha=0.7)
#     plt.xlabel("Energy")
#     plt.ylabel("Frequency")
#     plt.title("Histogram of Energy Values")
#     plt.grid(True)
#     plt.show()

#     return detected_phase_matrix, ground_state_energy

```

The histogram plot `plt.hist(energy_values, bins=50, color='purple', alpha=0.7)` line plots the histogram of the `energy_values`, is to give a visual idea of the distribution of energies throughout the simulation, helping to understand how frequently different energy levels occur during the process. The `bins` parameters can be adjusted for finner or coarser binning, depending on how spread the energies are. To ensure the histogram plot reflects the full distribution of energy values generated during the Monte Carlo process, it is placed after all iterations are done.

```

[31]: # # # Record the start time
# start_time = time.time()

# with tf.device('/GPU:0'):
#     monte_carlo_ground_state(detected_phase_matrix, I_target_shifted, xi, W,
#                               ↪ kx, ky, iterations = 100)

# # # Record the end time
# end_time = time.time()

# # # Calculate elapsed time
# elapsed_time = end_time - start_time

```

```

[32]: # # Print the elapsed time
# print(f"Computation Time: {elapsed_time:.4f} seconds")

```

The detected phase matrix in the final state is visualized to check if the spatial structure of the configuration could minimize the energy.

```

[33]: # # Create the pixel-like image
# # Create the plot
# fig, axes = plt.subplots(1, 2, figsize=(10, 5)) # gridspec_kw={'width_ratios':
#                               ↪ [1, 1.1]}

# # Plot the spin matrix on the left
# axes[0].imshow(target_phase_matrix, cmap='viridis')
# axes[0].set_title("Pixel Representation of \n Target Phase Matrix")

```



```

# axes[0].set_xlabel("Pixel Position x")
# axes[0].set_ylabel("Pixel Position y")

# # Plot the random phase matrix on the right
# cax = axes[1].imshow(detected_phase_matrix, cmap='viridis')
# axes[1].set_title("Pixel Representation of \n Final Detected Phase Matrix_
↳after iteration")
# axes[1].set_xlabel("Pixel Position x")
# axes[1].set_ylabel("Pixel Position y")

# # Add colorbar for the phase matrix plot
# cbar3 = fig.colorbar(cax, ax=axes[0])
# cbar4 = fig.colorbar(cax, ax=axes[1])

# # Set the colorbar ticks and labels
# cbar_ticks = [0, np.pi]
# cbar_labels = ['0', '$\pi$']

# cbar3.set_ticks(cbar_ticks)
# cbar3.set_ticklabels(cbar_labels)

# cbar4.set_ticks(cbar_ticks)
# cbar4.set_ticklabels(cbar_labels)

# # Show the plots
# plt.tight_layout()
# plt.show()

```

```
[34]: # print(detected_phase_matrix, target_phase_matrix)
```

To visualize how often specific energy values are distributed and how significant changes in energy changes appear over iteration, a histogram is plotted.

1.5 Simulate Annealing (SA)

Simulated Annealing is an extension to the traditional Monte Carlo Algorithm, where the system's exploration of phase space is improved by accepting the increased energy with some probability. The probability decrease over time, simulating a cooling process that helps the system settle into a global minimum rather than getting stuck in local minima. SA is directly integrated into the Monte Carlo method, which is to modify how decision about accepting or rejecting a move are made within the Monte Carlo loop. There is no need to separate it out into a different function since the primary change lies in adjusting how moves are accepted as a function of temperature.

Temperature `temperature` is introduced as a parameter, starting with a higher value and then gradually decrease it over iterations by multiplying it with another parameter `cooling_rate` after each iteration. The energy-lowering moves are only accepted in traditional Monte Carlo, while the both energy-lowering and energy-increasing moves with certain probability are also accepted with the integration of SA. The updated criterion is that the new configuration is accepted if the

new energy is lower. If the new energy is higher, it will be accepted with a probability given by $P = e^{-\frac{\Delta E}{T}}$, where ΔE is the energy difference and T is the temperature. To ensure the system can explore more configuration and escape local minima, the temperature decreases over time. However, eventually, the system behaves more like the traditional Monte Carlo method, only accepting downhill moves.

```
[35]: # def monte_carlo_with_simulated_annealing(detected_phase_matrix,
#       ↪ I_x_Target_shifted, xi, W, kx, ky, iterations=100, initial_temperature=1.5,
#       ↪ cooling_rate=0.99):
#     n = detected_phase_matrix.shape[0] # Size of the phase matrix

#     # Convert the initial phase matrix to a spin matrix
#     detected_spin_matrix = binary_phase_to_spin(detected_phase_matrix)

#     # Calculate and normalize the initial intensity distribution only once
#     I_x_Target = calculate_intensity_fourier(xi, target_spin_matrix, W, kx,
#       ↪ ky)
#     I_x_Target_normalized = (I_x_Target - np.min(I_x_Target)) / (np.
#       ↪ max(I_x_Target) - np.min(I_x_Target))
#     I_x_Target_shifted = np.fft.fftshift(I_x_Target_normalized)

#     # Calculate the detected intensity and normalize it
#     I_x_Detected = calculate_intensity_fourier(xi, detected_spin_matrix, W,
#       ↪ kx, ky)
#     I_x_Detected_shifted = np.fft.fftshift(I_x_Detected)
#     I_x_Detected_normalized = (I_x_Detected_shifted - np.
#       ↪ min(I_x_Detected_shifted)) / (np.max(I_x_Detected_shifted) - np.
#       ↪ min(I_x_Detected_shifted))

#     # Initial energy based on normalized intensity
#     current_energy = calculate_energy(I_x_Detected_normalized,
#       ↪ I_x_Target_shifted)

#     # Initialize ground state energy and energy tracking
#     ground_state_energy = current_energy
#     energy_values = [current_energy]
#     temperature = initial_temperature

#     for i in range(iterations):
#         # Randomly flip a small cluster of spins
#         new_phase_matrix = detected_phase_matrix.copy()
#         cluster_size = np.random.randint(1, 5)
#         random_indices = np.random.choice(n * n, cluster_size, replace=False)

#         for index in random_indices:
#             row, col = divmod(index, n)
```

```

#         new_phase_matrix[row, col] = np.pi - new_phase_matrix[row, col]
# Flip the phase

#         # Convert the new phase matrix to a spin matrix
#         new_spin_matrix = binary_phase_to_spin(new_phase_matrix)
#         new_intensity = calculate_intensity_fourier(xi, new_spin_matrix, W,
#         kx, ky)
#         new_intensity_shifted = np.fft.fftshift(new_intensity)

#         # Normalize the new intensity
#         new_intensity_normalized = (new_intensity_shifted - np.
# min(new_intensity_shifted)) / (np.max(new_intensity_shifted) - np.
# min(new_intensity_shifted))

#         # Calculate the new energy
#         new_energy = calculate_energy(new_intensity_normalized,
# I_x_Target_shifted)
#         energy_diff = new_energy - current_energy
#         energy_values.append(new_energy)

#         # Simulated annealing acceptance condition
#         if new_energy < current_energy:
#             detected_phase_matrix = new_phase_matrix
#             current_energy = new_energy
#         else:
#             acceptance_prob = np.exp(-energy_diff / temperature)
#             if np.random.rand() < acceptance_prob:
#                 detected_phase_matrix = new_phase_matrix
#                 current_energy = new_energy

#         if current_energy < ground_state_energy:
#             ground_state_energy = current_energy

#         # Cooling
#         temperature *= cooling_rate

#         # Print progress every 5 iterations
#         if i % 5 == 0:
#             print(f"Iteration {i}, Energy: {current_energy}, Ground State
# Energy: {ground_state_energy}, Temperature: {temperature}")

#         return detected_phase_matrix, ground_state_energy, energy_values,
# new_intensity_normalized

```

```

[36]: def monte_carlo_with_feedback(detected_phase_matrix, I_x_Target_shifted, xi, W,
# kx, ky, iterations=100, initial_temperature=1.5, cooling_rate=0.99):
#         n = detected_phase_matrix.shape[0] # Size of the phase matrix

```

```

detected_spin_matrix = binary_phase_to_spin(detected_phase_matrix)

# Normalize the target intensity
I_x_Target_normalized = (I_x_Target_shifted - np.min(I_x_Target_shifted)) /
↪(np.max(I_x_Target_shifted) - np.min(I_x_Target_shifted))

# Compute initial detected intensity and normalize it
I_x_Detected = calculate_intensity_fourier(xi, detected_spin_matrix, W, kx,
↪ky)
I_x_Detected_normalized = (I_x_Detected - np.min(I_x_Detected)) / (np.
↪max(I_x_Detected) - np.min(I_x_Detected))

current_energy = calculate_energy(I_x_Detected_normalized,
↪I_x_Target_normalized)
ground_state_energy = current_energy

# Track energy values for visualization
energy_values = [current_energy]

temperature = initial_temperature

for i in range(iterations):
    # Randomly flip a small cluster of spins
    new_phase_matrix = detected_phase_matrix.copy()
    cluster_size = np.random.randint(1, 5)
    random_indices = np.random.choice(n * n, cluster_size, replace=False)

    for index in random_indices:
        row, col = divmod(index, n)
        new_phase_matrix[row, col] = np.pi - new_phase_matrix[row, col] #
↪Flip the phase

    # Convert the new phase matrix to a spin matrix and calculate new
↪intensity
    new_spin_matrix = binary_phase_to_spin(new_phase_matrix)
    new_intensity = calculate_intensity_fourier(xi, new_spin_matrix, W, kx,
↪ky)
    new_intensity_normalized = (new_intensity - np.min(new_intensity)) /
↪(np.max(new_intensity) - np.min(new_intensity))

    # Calculate new energy
    new_energy = calculate_energy(new_intensity_normalized,
↪I_x_Target_normalized)
    energy_diff = new_energy - current_energy

```

```

    # Accept the new configuration if it reduces energy, or with some
    ↪probability if it increases energy (for simulated annealing)
    if new_energy < current_energy or np.random.rand() < np.
    ↪exp(-energy_diff / temperature):
        detected_phase_matrix = new_phase_matrix
        current_energy = new_energy

    # Track the ground state energy
    if current_energy < ground_state_energy:
        ground_state_energy = current_energy

    # Append the current energy to the list for tracking
    energy_values.append(current_energy)

    # Cooling step
    temperature *= cooling_rate

    # Optionally print progress every few iterations
    if i % 5 == 0:
        print(f"Iteration {i}, Energy: {current_energy:.6f}, Ground State
    ↪Energy: {ground_state_energy:.6f}, Temperature: {temperature:.6f}")

    # Return all 4 values: detected_phase_matrix, ground_state_energy,
    ↪energy_values, new_intensity_normalized
    return detected_phase_matrix, ground_state_energy, energy_values,
    ↪new_intensity_normalized

```

```

[37]: # Visualization function to keep it separate from the main algorithm
def visualize_results(I_x_Target_shifted, new_intensity_normalized,
    ↪energy_values, iterations):
    # Apply FFT shift to the final intensity for visualization
    I_x_Final_Detected_shifted = np.fft.fftshift(new_intensity_normalized)

    # Create a figure with two subplots side by side for the intensity patterns
    fig, axes = plt.subplots(1, 2, figsize=(12, 6))

    # Plot the target intensity pattern on the left
    im1 = axes[0].imshow(I_x_Target_shifted, cmap='viridis')
    axes[0].set_title("Target Intensity Pattern $I_T(x)$ (Shifted)")
    axes[0].set_xlabel("Pixel Position X")
    axes[0].set_ylabel("Pixel Position Y")
    fig.colorbar(im1, ax=axes[0], label="Intensity")

    # Plot the detected intensity pattern on the right (after iterations)
    im2 = axes[1].imshow(I_x_Final_Detected_shifted, cmap='viridis')
    axes[1].set_title(f"Detected Intensity $I_D(x)$ (Shifted) at Iteration
    ↪{iterations}")

```

```

axes[1].set_xlabel("Pixel Position X")
axes[1].set_ylabel("Pixel Position Y")
fig.colorbar(im2, ax=axes[1], label="Intensity")

# Adjust layout and display the plots
plt.tight_layout()
plt.show()

# Calculate the energy differences between iterations
energy_diffs = [energy_values[i] - energy_values[i-1] for i in range(1,
↪len(energy_values))]

# Plot energy differences and values over iterations
plt.figure(figsize=(10, 6))

# Plot 1: Energy differences
plt.subplot(2, 1, 1)
plt.plot(energy_diffs, label="Energy Difference", color='b')
plt.xlabel("Iteration")
plt.ylabel("Energy Difference")
plt.title("Energy Difference Between Iterations")
plt.grid(True)
plt.legend()

# Plot 2: Energy values
plt.subplot(2, 1, 2)
plt.plot(energy_values, label="Energy Values", color='g')
plt.xlabel("Iteration")
plt.ylabel("Energy")
plt.title("Energy Values Over Iterations")
plt.grid(True)
plt.legend()

plt.tight_layout()
plt.show()

# Plot the histogram of energy values
plt.figure(figsize=(10, 6))
plt.hist(energy_values, bins=50, color='purple', alpha=0.7)
plt.xlabel("Energy")
plt.ylabel("Frequency")
plt.title("Histogram of Energy Values")
plt.grid(True)
plt.show()

```

```

[38]: # # Record the start time
start_time_SA = time.time()

```

```

with tf.device('/GPU:0'):
    detected_phase_matrix, ground_state_energy, energy_values,
    ↪new_intensity_normalized = monte_carlo_with_feedback(detected_phase_matrix,
    ↪I_x_Target_shifted, xi, W, kx, ky, iterations=200, initial_temperature=2.0,
    ↪cooling_rate=0.99)
    detected_phase_matrix, ground_state_energy, energy_values,
    ↪new_intensity_normalized
    visualize_results(I_x_Target_shifted, new_intensity_normalized,
    ↪energy_values, iterations=200)
# # Record the end time
end_time_SA = time.time()

# # Calculate elapsed time
elapsed_time_SA = end_time_SA - start_time_SA
elapsed_time_SA

```

```

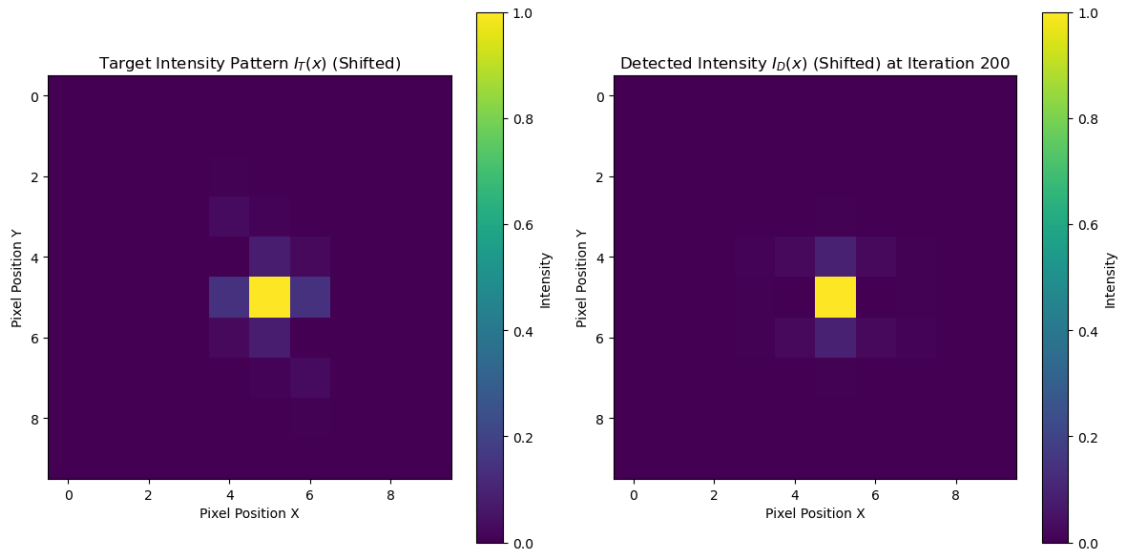
2024-09-19 14:58:25.290351: I metal_plugin/src/device/metal_device.cc:1154]
Metal device set to: Apple M2 Pro
2024-09-19 14:58:25.290370: I metal_plugin/src/device/metal_device.cc:296]
systemMemory: 32.00 GB
2024-09-19 14:58:25.290375: I metal_plugin/src/device/metal_device.cc:313]
maxCacheSize: 10.67 GB
2024-09-19 14:58:25.290390: I
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:305]
Could not identify NUMA node of platform GPU ID 0, defaulting to 0. Your kernel
may not have been built with NUMA support.
2024-09-19 14:58:25.290400: I
tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271]
Created TensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 0
MB memory) -> physical PluggableDevice (device: 0, name: METAL, pci bus id:
<undefined>)
2024-09-19 14:58:25.566406: I
tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:117]
Plugin optimizer for device_type GPU is enabled.

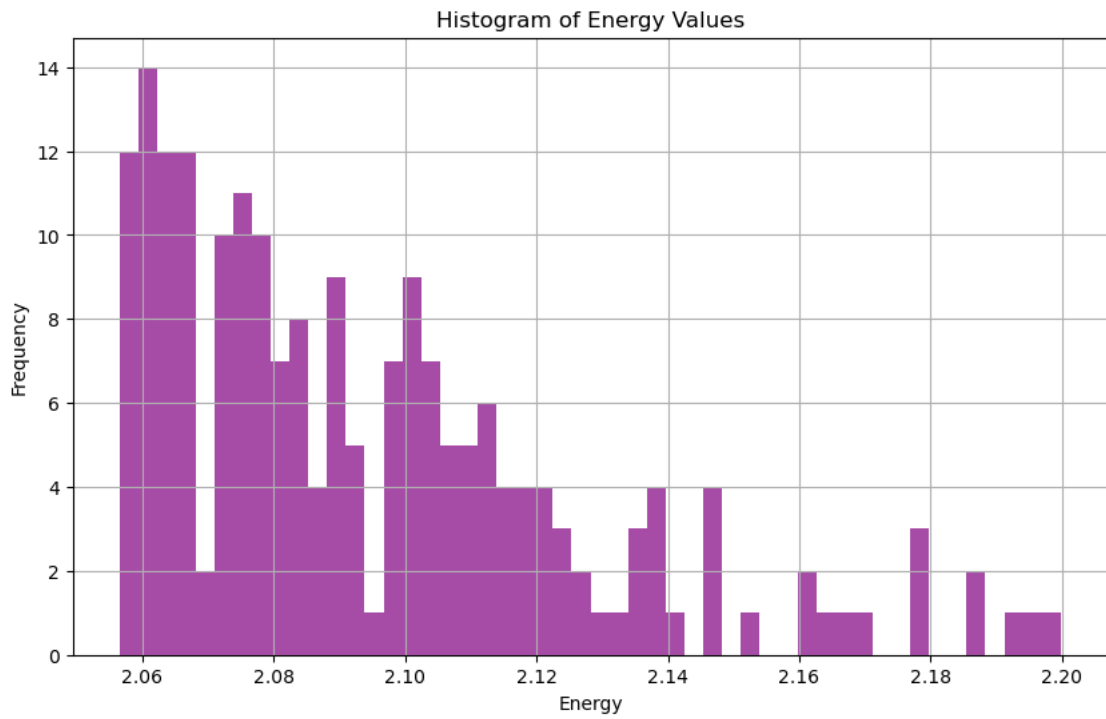
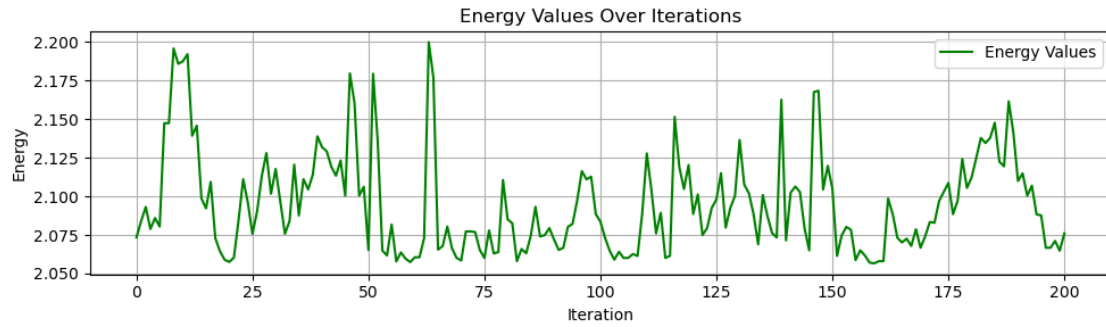
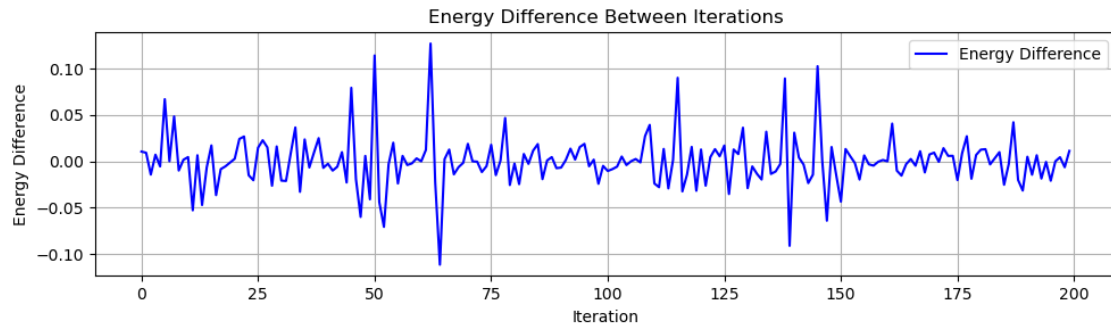
Iteration 0, Energy: 2.083913, Ground State Energy: 2.073424, Temperature:
1.980000
Iteration 5, Energy: 2.147363, Ground State Energy: 2.073424, Temperature:
1.882960
Iteration 10, Energy: 2.192050, Ground State Energy: 2.073424, Temperature:
1.790677
Iteration 15, Energy: 2.109300, Ground State Energy: 2.073424, Temperature:
1.702916
Iteration 20, Energy: 2.060263, Ground State Energy: 2.057515, Temperature:
1.619456
Iteration 25, Energy: 2.090252, Ground State Energy: 2.057515, Temperature:
1.540086

```

Iteration 30, Energy: 2.096909, Ground State Energy: 2.057515, Temperature:
1.464607
Iteration 35, Energy: 2.111150, Ground State Energy: 2.057515, Temperature:
1.392826
Iteration 40, Energy: 2.129190, Ground State Energy: 2.057515, Temperature:
1.324564
Iteration 45, Energy: 2.179607, Ground State Energy: 2.057515, Temperature:
1.259647
Iteration 50, Energy: 2.179346, Ground State Energy: 2.057515, Temperature:
1.197912
Iteration 55, Energy: 2.057842, Ground State Energy: 2.057515, Temperature:
1.139202
Iteration 60, Energy: 2.060529, Ground State Energy: 2.057400, Temperature:
1.083370
Iteration 65, Energy: 2.067799, Ground State Energy: 2.057400, Temperature:
1.030274
Iteration 70, Energy: 2.077279, Ground State Energy: 2.057400, Temperature:
0.979781
Iteration 75, Energy: 2.077920, Ground State Energy: 2.057400, Temperature:
0.931762
Iteration 80, Energy: 2.082624, Ground State Energy: 2.057400, Temperature:
0.886096
Iteration 85, Energy: 2.093237, Ground State Energy: 2.057400, Temperature:
0.842668
Iteration 90, Energy: 2.065271, Ground State Energy: 2.057400, Temperature:
0.801369
Iteration 95, Energy: 2.116322, Ground State Energy: 2.057400, Temperature:
0.762094
Iteration 100, Energy: 2.073138, Ground State Energy: 2.057400, Temperature:
0.724744
Iteration 105, Energy: 2.060132, Ground State Energy: 2.057400, Temperature:
0.689224
Iteration 110, Energy: 2.103905, Ground State Energy: 2.057400, Temperature:
0.655446
Iteration 115, Energy: 2.151499, Ground State Energy: 2.057400, Temperature:
0.623322
Iteration 120, Energy: 2.101207, Ground State Energy: 2.057400, Temperature:
0.592773
Iteration 125, Energy: 2.115009, Ground State Energy: 2.057400, Temperature:
0.563721
Iteration 130, Energy: 2.107583, Ground State Energy: 2.057400, Temperature:
0.536093
Iteration 135, Energy: 2.087236, Ground State Energy: 2.057400, Temperature:
0.509820
Iteration 140, Energy: 2.102343, Ground State Energy: 2.057400, Temperature:
0.484833
Iteration 145, Energy: 2.167505, Ground State Energy: 2.057400, Temperature:
0.461072

Iteration 150, Energy: 2.061414, Ground State Energy: 2.057400, Temperature: 0.438475
 Iteration 155, Energy: 2.065019, Ground State Energy: 2.057400, Temperature: 0.416985
 Iteration 160, Energy: 2.057993, Ground State Energy: 2.056600, Temperature: 0.396549
 Iteration 165, Energy: 2.072603, Ground State Energy: 2.056600, Temperature: 0.377114
 Iteration 170, Energy: 2.083406, Ground State Energy: 2.056600, Temperature: 0.358631
 Iteration 175, Energy: 2.088444, Ground State Energy: 2.056600, Temperature: 0.341055
 Iteration 180, Energy: 2.124750, Ground State Energy: 2.056600, Temperature: 0.324340
 Iteration 185, Energy: 2.122385, Ground State Energy: 2.056600, Temperature: 0.308444
 Iteration 190, Energy: 2.114857, Ground State Energy: 2.056600, Temperature: 0.293327
 Iteration 195, Energy: 2.066690, Ground State Energy: 2.056600, Temperature: 0.278951





[38] : 50.394627809524536

```
[40]: # Assuming you have already calculated 'new_intensity' in the Monte Carlo
      ↪ simulated annealing
I_x_Final_Detected_shifted = np.fft.fftshift(new_intensity_normalized) #
      ↪ Final detected intensity after the iterations
# I_x_Detected_shifted = np.fft.fftshift(I_x_Detected_normalized) # Detected
      ↪ intensity before iterations (initial state)

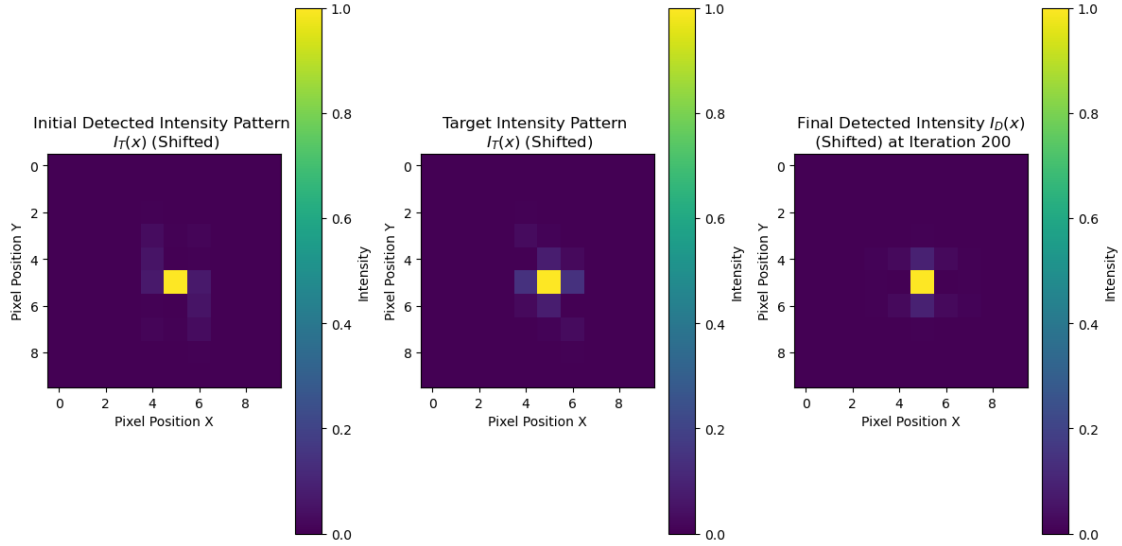
# Create a figure with two subplots side by side for the intensity patterns
fig, axes = plt.subplots(1, 3, figsize=(12, 6))

# Plot the initial detected intensity pattern on the left
im0 = axes[0].imshow(I_x_Detected_shifted, cmap='viridis')
axes[0].set_title("Initial Detected Intensity Pattern \n $I_T(x)$ (Shifted)")
axes[0].set_xlabel("Pixel Position X")
axes[0].set_ylabel("Pixel Position Y")
fig.colorbar(im0, ax=axes[0], label="Intensity")

# Plot the target intensity pattern on the left
im1 = axes[1].imshow(I_x_Target_shifted, cmap='viridis')
axes[1].set_title("Target Intensity Pattern \n $I_T(x)$ (Shifted)")
axes[1].set_xlabel("Pixel Position X")
axes[1].set_ylabel("Pixel Position Y")
fig.colorbar(im1, ax=axes[1], label="Intensity")

# Plot the final detected intensity pattern on the right (after iterations)
im2 = axes[2].imshow(I_x_Final_Detected_shifted, cmap='viridis')
axes[2].set_title(f"Final Detected Intensity $I_D(x)$ \n (Shifted) at Iteration
      ↪ {200}")
axes[2].set_xlabel("Pixel Position X")
axes[2].set_ylabel("Pixel Position Y")
fig.colorbar(im2, ax=axes[2], label="Intensity")

# Adjust layout and display the plots
plt.tight_layout()
plt.show()
```



From the generated plot, we observe that there are still some mismatch between target and the final intensity patterns. Despite the iterations, the final detected intensity still resembles the initial detected intensity in certain ways. Also, there is a substantial difference between the magnitude of the final detected intensity and that of the target intensity.

The close proximity between the initial and final detected pattern suggests that the optimization algorithm may not be exploring the search space effectively. One of the reason could be the temperature schedule or the cooling rate in SA process may be too conservative, leading to insufficient exploration or early convergence to a local minimum. If the function does not properly account for the magnitude scaling, the optimization may not be sensitive enough to bridge this gap.

```
[ ]: I_x_Final_Detected_shifted, I_x_Detected_shifted, I_x_Target_shifted
```

It is possible that the standard Monte Carlo method might find a lower energy minimum. The methods are stochastic such that the results can vary between runs due to the randomness in selecting which spins to flip. If the initial phase matrices for both methods are different, they may explore different paths through the energy landscape. Besides, if the temperature decreases too quickly or the cooling rate is too large in the SA method, the system might not explore enough of the energy landscape to find the global minimum, of which have to be carefully tuned. Added to it, SA might need more iterations to outperform the standard Monte Carlo method.

The flipping small clusters of phases (random clusters of size 1 to 5) may help explore local neighbourhoods, but it may not be sufficient to explore larger regions of the solution space. Besides, the cooling rate might be too fast or too slow depending on the nature of the problem. If the temperature drops too quickly, the algorithm can get stuck in local minima. It is advisable to introduce both adaptive cluster size and cooling rate that starts small and increases gradually as the algorithm progresses based on the energy landscape.

```
[ ]: # import tensorflow as tf
      # import numpy as np
```

```

# # Specify the directory to save the profiling data
# logdir = 'profiler_logs'

# # Define a function to compute energy (placeholder example)
# def compute_energy(state):
#     # Replace with your actual energy calculation (e.g., Ising model, etc.)
#     return tf.reduce_sum(state ** 2)

# # Initialize random state (could represent a physical system's configuration)
# def random_state(n):
#     return tf.random.uniform((n, n))

# # Simulated Annealing process
# def simulated_annealing(n, iterations):
#     state = random_state(n)
#     energy = compute_energy(state)

#     for i in range(iterations):
#         # Propose a new state (this is a placeholder; replace with actual
#         ↪ transition logic)
#         new_state = state + tf.random.normal(state.shape) * 0.1
#         new_energy = compute_energy(new_state)

#         # Accept or reject the new state based on energy difference
#         if new_energy < energy:
#             state = new_state
#             energy = new_energy

#     return energy

# # Start profiling
# tf.profiler.experimental.start(logdir)

# # Run the SA process
# with tf.device('/GPU:0'):
#     final_energy = simulated_annealing(n=10, iterations=1000)

# # Stop profiling
# tf.profiler.experimental.stop()

# # Print final energy
# print("Final Energy:", final_energy)

# # # Load TensorBoard to visualize the profiling results
# # %load_ext tensorboard
# # %tensorboard --logdir=profiler_logs

```

2 Draft

2.1 Using GPU for Computation

```
[ ]: # import tensorflow as tf
      # print("Num GPUs Available: ", len(tf.config.list_physical_devices('GPU')))
```

```
[ ]: import tensorflow as tf

def calculate_intensity_fourier_tf(xi, spin_matrix, W, kx, ky):
    # Ensure that inputs are converted to TensorFlow tensors
    spin_matrix = tf.convert_to_tensor(spin_matrix, dtype=tf.float32)
    kx = tf.convert_to_tensor(kx, dtype=tf.float32)
    ky = tf.convert_to_tensor(ky, dtype=tf.float32)

    # Initialize E_k in the Fourier domain (complex field)
    E_k = tf.zeros_like(kx, dtype=tf.complex64)

    # Perform operations (replace NumPy ops with TensorFlow-compatible ones)
    # This is a simplified version; you will need to convert all NumPy
    ↪operations to TensorFlow

    E_x = tf.signal.ifft2d(E_k)
    I_x = tf.abs(E_x) ** 2
    return I_x
```

```
[ ]: calculate_intensity_fourier_tf(xi, spin_matrix, W, kx, ky)
```

```
[ ]: # import torch

    # # Check if GPU is available and set the device
    # device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    # print(f"Using device: {device}")

    # # Example matrix operation to calculate energy
    # def compute_energy(state):
    #     # Example energy computation using a Hamiltonian matrix
    #     H = torch.randn((1000, 1000), device=device) # Hamiltonian matrix on GPU
    #     energy = torch.matmul(state.T, torch.matmul(H, state))
    #     return energy

    # # Initialize state (random vector)
    # state = torch.randn((1000, 1), device=device) # State vector on GPU

    # # Perform energy calculation
    # energy = compute_energy(state)
    # print(f"Energy: {energy.item()}")
```

```
[ ]: # # Example Monte Carlo loop
# for _ in range(10000):
#     # Generate a new random state (on GPU)
#     new_state = torch.randn((1000, 1), device=device)

#     # Compute the energy of the new state
#     new_energy = compute_energy(new_state)

#     # Acceptance condition for simulated annealing (simple example)
#     if torch.rand(1, device=device) < torch.exp(-new_energy / 100):
#         state = new_state # Accept the new state if condition is met
```

2.1.1 Practical Consideration

$\xi = 0.5$ is quite small. Since $I(x) \propto \xi^2$, this value will reduce the overall intensity by a factor of $0.5^2 = 0.25$.

W may lead to a very narrow sinc function in the spatial frequency domain, especially if $k - k_j$ is large. This could result in very small values of E_k and hence small $I(x)$.

5. Decoding: The final phase modulations are converted back into binary spin values, representing the solution to the optimization problem.

```
[ ]: # import numpy as np
# import matplotlib.pyplot as plt

# # Example spatial frequency grid
# k = np.linspace(-10, 10, 400)
# W = 1 # Pixel width

# # Compute sinc functions
# sinc_k = np.sinc(W * k)

# # Plot the sinc function
# plt.plot(k, sinc_k)
# plt.title("Normalized Sinc Function")
# plt.xlabel("Frequency")
# plt.ylabel("Amplitude")
# plt.grid(True)
# plt.show()
```

```
[ ]: # import numpy as np
# import matplotlib.pyplot as plt

# # Example parameters
# n = 2
# xi = 0.7
# W = 10e-6 # Pixel width
```

```

# pixel_pitch = 20e-6
# wavelength = 532e-9
# num_points = 256 # Adjust as needed

# # phase_matrix = np.array([
# #     [0, 0, np.pi, 0],
# #     [np.pi, 0, 0, np.pi],
# #     [0, np.pi, np.pi, 0],
# #     [np.pi, 0, 0, np.pi]
# # ])

# # Generate example spin matrix
# spin_matrix = np.array([[1, -1], [-1, 1]])

# binary_phase_to_spin(phase_matrix)

# # Calculate spatial frequency grid
# kx, ky = calculate_spatial_frequency_grid(pixel_pitch, wavelength, num_points)

# # Calculate intensity pattern
# I_x = calculate_intensity_fourier(xi, spin_matrix, W, kx, ky)

# # Plot the intensity
# plt.imshow(I_x, cmap='gray')
# plt.title("Intensity Pattern")
# plt.colorbar(label="Intensity")
# plt.show()

```

[]:

[]: